

Momo Desk Buddy: System Design and Functionality

1 Project Overview

Momo is a personal desk buddy robot designed to help users stay organized, productive, and engaged while working at their desk. It combines a touch interface, voice commands, live weather, task management, Pomodoro timers, reminders, music playback, computer vision, and Arduino-controlled movement. The goal of Momo is to create a small desktop assistant that can respond to the user, display useful information, and physically turn toward a person using a webcam and servo motor.

The system is built with a Flask backend, a web-based touch interface, Python modules, OpenCV computer vision, JSON task storage, speech recognition, text-to-speech, music playback tools, and an Arduino servo system. These parts work together through a central shared state dictionary that stores the current screen, tasks, weather, timer, reminders, presence detection, sleep mode, music status, and servo position.

2 Main Architecture

Momo uses a Flask backend as the main control center. The robot takes input from three main sources: voice commands, the touch interface, and webcam/OpenCV detection. Voice commands are processed by the command parser, which determines what the user wants Momo to do. Touch input comes from the web interface and is sent to Flask API routes. Camera input is processed by an OpenCV thread that detects people and tracks their position.

Once an input is received, the backend updates a shared state dictionary. This shared state stores important information such as the current screen, voice status, robot face expression, weather data, task list, timer information, reminders, presence status, sleep mode, music status, and servo angle. The frontend repeatedly reads from this state through the Flask API and updates the display. Other parts of the system, such as speech output, music playback, weather requests, and servo movement, also respond based on the shared state.

This design keeps the project organized because all features communicate through one central source of truth instead of each part managing its own separate data.

3 Touch Interface

Momo has a web-based touch interface designed for a small screen. The interface includes a sidebar and a main display area. The sidebar shows Momo’s face, name, current time, date, weather summary, task count, navigation buttons, presence status, microphone status, sleep status, and music controls.

The main screen can switch between different views. The clock screen shows the current time, date, robot face, and Pomodoro timer controls. The weather screen shows the current weather for New Haven, including temperature and condition. The tasks screen displays today’s tasks and lets the user add, complete, or remove tasks.

The touch interface makes Momo usable even without voice commands. Users can navigate between screens, start timers, manage tasks, check weather, and stop music directly from the display.

4 Voice Control

Momo uses voice control so the user can interact with it naturally. The voice system listens for the wake word “Momo.” After the wake word is detected, Momo records the rest of the command and converts the speech into text. The command is then sent to the command parser.

The command parser identifies the user’s intent. For example, it can recognize when the user wants the weather, the time, a task added, a timer started, a reminder set, a joke told, music played, or the servo moved. The system uses keyword-based parsing. OpenAI’s API was attempted to be used, but it failed because we have no money.

After the intent is identified, the backend calls `execute_action()`, which updates the shared state and triggers the correct response. Momo can then speak back to the user using text-to-speech.

5 Clock and Date

Momo displays the current time and date on both the main screen and sidebar. The backend sends the current time and date to the frontend through the state API, and the frontend updates the display.

Users can also ask Momo for the time using a voice command. When Momo detects a time request, it switches to the clock screen and speaks the current time out loud. This makes the clock feature available through both the visual interface and voice interaction.

6 Weather System

Momo fetches live weather for New Haven using an online weather service. When the user asks for the weather or taps the weather button, the backend sends a request for current weather data. It extracts the temperature in Fahrenheit and the weather condition, then stores that information in the shared state.

The weather information is displayed on the weather screen and in the sidebar. Momo also gives spoken clothing recommendations based on the temperature and condition. For example, if it is cold, Momo may suggest a warm jacket. If it is raining, Momo may recommend bringing an umbrella. If it is sunny, Momo may suggest sunglasses.

This makes the weather feature more useful because Momo does not just report the temperature, it also gives practical advice.

7 Task Management

Momo includes a task manager for daily productivity. Users can view today's tasks, add new tasks, mark tasks as complete, and remove tasks. Tasks are stored in a JSON file, which allows them to persist after the program restarts.

The backend loads the tasks from the JSON file when the program starts. When the user adds, completes, or removes a task, the backend updates the task list and saves the new version back to the file. This keeps the data simple and easy to manage.

Tasks can be controlled through both the touch interface and voice commands. For example, a user can say "Momo, add finish homework" or "Momo, complete check email." If the user gives an incomplete command, Momo can ask a follow-up question to get the missing task name.

8 Pomodoro Timer

Momo has a built-in Pomodoro timer system. The interface includes quick-start buttons for a 25-minute focus session and a 5-minute break. When a timer is active, the screen shows the timer label, countdown, progress bar, and cancel button.

The timer information is stored in the shared state dictionary. It includes whether the timer is active, how many seconds are left, the total duration, and the timer label. A background timer thread runs every second and decreases the countdown while the timer is active.

When the timer reaches zero, Momo marks the timer as complete, changes its expression, and speaks a message to let the user know the session is finished. Because the timer runs in its own thread, it can keep counting down while the user continues using other features.

9 Spoken Reminders

Momo can set spoken reminders from voice commands. For example, the user can say, “Momo, remind me to drink water in 30 minutes.” The backend extracts the reminder text and the time duration, then stores the reminder in the shared state.

A reminder-checking thread runs in the background and checks whether any reminders are due. When the current time reaches the reminder time, Momo speaks the reminder out loud and marks it as fired. This allows Momo to act like a small productivity assistant that helps the user remember important tasks.

10 Daily Summary

Momo can give a daily summary that combines multiple features into one response. When the user asks for a daily briefing, Momo reports the current time, current weather, clothing recommendation, and pending tasks.

If there are no pending tasks, Momo tells the user they are all caught up. If there are a few tasks, Momo lists them. If there are many tasks, it summarizes the number and mentions the first few. This feature shows how Momo combines separate systems into one useful interaction.

11 Jokes and Personality

Momo has a cheerful robot personality. It can tell clean jokes, especially robot, computer, and technology-themed jokes. This makes the robot feel more friendly and engaging.

The personality system is also used in spoken responses. Momo gives encouraging feedback when tasks are completed, timers are started, or daily summaries are given. This helps the project feel more like a desk companion instead of just a utility program.

12 Sleep and Wake Mode

Momo includes sleep and wake modes. When the user says a sleep command such as “go to sleep” or “goodbye,” Momo enters sleep mode and changes its face to a sleeping expression. While asleep, Momo ignores most commands.

To wake Momo, the user says the wake word with a wake command such as “Momo, wake up.” Momo then exits sleep mode and becomes ready to listen again. Sleep mode is useful because it gives the user control over when Momo should be active.

13 Music Playback System

Momo's music feature allows the user to ask for a song using a voice command, such as "Momo, play music" or "Momo, play [song name]." The voice command is processed by the command parser, which identifies the request as a `play_music` intent. The parser extracts the song or artist name from the user's command and sends it to the backend action system.

If a song name is found, Momo first gives a spoken confirmation such as "On it! Playing [song name]." After the text-to-speech response finishes, the music system starts in a separate thread so the rest of the program can continue running normally.

The music playback function first stops any currently playing music to avoid overlapping audio. Then, it searches YouTube for the requested song using `yt-dlp`. Instead of downloading a full video file, the system streams the best available audio. The audio stream is passed into `ffmpeg`, which converts it into a raw audio format that can be played by the Raspberry Pi's audio system. The converted audio is then sent to `paplay` or `aplay` for playback through the speaker.

While music is playing, Momo stores the song name in the shared state dictionary. This allows the touch interface to show a music banner with the current song and a stop button. The stop button sends a request to the backend, which stops the active music process and clears the song from the shared state.

Momo can also stop music through a voice command. If the user says something like "stop music," the backend kills the active music process and removes the music banner from the interface. A monitoring thread watches the music process, so if the song ends naturally, Momo automatically clears the music state.

This design lets music play in the background without freezing the voice assistant, timer, reminders, or web interface.

14 More about Music Streaming

The music system does not download a full audio file before playing it. Instead, `yt-dlp` searches YouTube for the requested song and streams the best available audio directly to standard output using `-o -`. This means there is no saved intermediate music file. The first output from `yt-dlp` is an audio stream, usually in a compressed format such as WebM/Opus or M4A/AAC.

```
yt-dlp -f bestaudio --no-playlist "ytsearch1:song name" -o -
```

The streamed audio is then piped into `ffmpeg`. In the command, `pipe:0` tells `ffmpeg` to read from standard input instead of reading from a file. `ffmpeg` decodes the compressed audio stream and converts it into raw PCM audio.

```
ffmpeg -i pipe:0 -f s16le -ar 44100 -ac 2 pipe:1
```

The conversion options mean:

- `-f s16le`: outputs raw 16-bit little-endian PCM audio
- `-ar 44100`: sets the sample rate to 44.1 kHz
- `-ac 2`: outputs stereo audio
- `pipe:1`: sends the converted audio to standard output

Finally, the raw PCM audio is piped into `paplay` or `aplay`, which sends the sound to the speaker.

```
paplay --raw --channels=2 --rate=44100 --format=s16le
```

The full process is:

```
YouTube compressed audio stream
-> yt-dlp streams audio to stdout
-> ffmpeg reads from pipe:0
-> ffmpeg decodes and converts to raw PCM audio
-> paplay/aplay plays audio through the speaker
```

This design avoids saving temporary music files and allows playback to begin while the audio is still being streamed and converted.

Note this is where DAC happens:

`paplay/aplay` → audio hardware → speaker

The Raspberry Pi/audio hardware converts the digital PCM audio into an analog electrical signal that drives the speaker.

15 Camera Tracking System

Momo uses a webcam and OpenCV to detect when a person is at the desk and track the person's position. The camera system runs in its own background thread called `camera_thread`. This lets Momo continuously process video frames while the rest of the program continues running.

The camera thread opens the webcam using OpenCV and captures frames from the video feed. Each frame is resized to make detection faster. The frame is then converted to grayscale, and histogram equalization is applied to improve contrast. This helps the detection system work better in different lighting conditions.

Momo first tries to detect a face using OpenCV's Haar cascade face detector. If no face is detected, the system can also try full-body detection as a fallback. This makes the system more reliable if the person's face is turned away or not clearly visible.

When a face or body is detected, Momo updates the shared state to show that someone is present. To prevent flickering, the code does not immediately mark the user as absent after one failed frame. Instead, it waits several frames before deciding that the person is no longer there.

For tracking, Momo calculates the horizontal center of the detected face and compares it to the horizontal center of the camera frame. If the face is close to the center, the servo does not move. If the face is too far left or right, the system calculates an offset and moves the servo in that direction.

A dead zone is used to prevent jitter. This concept is taken from binary input of 0s and 1s and how there is a margin of error in between. This means small face movements near the center are ignored so the servo does not constantly shake. If the face is farther from the center, the servo moves more. This creates smoother tracking because Momo turns more when the person is far off-center and less when the person is almost centered.

The camera tracking system only moves the servo when Momo is awake. If Momo is in sleep mode, the camera may still detect presence, but the robot does not keep turning unnecessarily.

16 Arduino Servo Movement

Momo's physical turning motion is controlled by an Arduino and a servo motor. The Arduino runs a simple program that listens for angle values sent over a serial connection. The servo is attached to pin 9, starts at 90 degrees, and can move between 0 and 180 degrees.

When the Arduino receives a number through serial, it checks whether the number is between 0 and 180. If the value is valid, the Arduino writes that angle to the servo and prints a confirmation message. This keeps the Arduino code simple and focused on low-level motor control.

On the Python side, the Flask backend connects to the Arduino using the `pyserial` library. During startup, the backend opens the serial port, waits briefly for the Arduino to reset, and sends the starting angle of 90 degrees. The backend also stores the servo's connection status and current angle in the shared state dictionary.

Servo movement can happen in two ways. First, the user can directly control the servo with voice commands such as "turn left," "turn right," or "center." The command parser converts these phrases into either a relative movement or an absolute angle. For example, "center" becomes 90 degrees.

Second, the OpenCV camera thread can automatically move the servo while tracking a face. If the face appears to the left or right of the frame center, the backend calculates a new servo angle and sends it to the Arduino as a number followed by a newline.

The Python servo function includes safety checks. It clamps the angle so it always stays between the minimum and maximum values. It also ignores very tiny changes to reduce

jitter. If a serial write fails, the backend marks the servo as disconnected so the program does not keep sending commands to unavailable hardware.

This setup separates responsibilities clearly. Python handles vision, logic, command parsing, and angle calculation. The Arduino handles the direct servo movement.

17 Concurrency and Locking

Momo uses concurrency so multiple features can run at the same time. Voice listening, timer countdowns, reminder checks, camera tracking, and Flask/API updates all run in separate threads or processes. This prevents one feature from blocking the others.

For example, the timer can continue counting down while the camera tracks a person. The Flask interface can keep updating while the reminder thread checks for reminders. The music system can play audio while the main program continues responding to input.

Since these threads all interact with the same shared state dictionary, Momo uses `state_lock` to protect shared data. Whenever a thread needs to read or update shared information, it does so inside:

```
with state_lock:
    # read or update shared state here
```

This ensures that only one thread can access that critical section at a time. The lock prevents race conditions, such as two threads trying to update the robot's face expression, timer state, or task list at the same time.

The code also avoids holding the lock during slower operations. Weather API requests, speech playback, music playback, subprocess calls, and camera processing happen outside the locked section whenever possible. This prevents the system from freezing while waiting for slow tasks to finish.

Overall, concurrency lets Momo run many background systems at once, while locking keeps the shared state safe and consistent.

18 Flask API Routes

The Flask backend provides API routes that connect the frontend to the backend system. The home route serves the main interface. The state route sends the current shared state to the frontend so the screen can update.

Other routes allow the frontend to change screens, fetch weather, add tasks, complete tasks, remove tasks, start timers, cancel timers, and stop music. These routes update the same shared state used by the voice system and background threads.

This API structure lets the touch interface and voice assistant control the same robot behavior.

19 Example Command Flow: Weather Request

A weather request shows how different parts of Momo work together.

First, the user says, “Momo, what’s the weather?” The voice loop records the audio and transcribes it into text. The command parser identifies the intent as `get_weather`. Then `execute_action()` fetches weather data from the weather service.

After the weather data is received, the backend updates the shared state with the new temperature and condition. It also changes the current screen to the weather screen. The frontend reads the updated state and displays the weather information. Finally, Momo speaks the weather and clothing recommendation out loud.

This shows how Momo connects voice input, command parsing, external data, shared state, frontend updates, and spoken output.

20 Example Command Flow: Music Request

A music request follows a similar structure but uses external audio tools.

First, the user says, “Momo, play [song name].” The voice loop transcribes the command, and the parser identifies it as a `play_music` intent. The backend extracts the song name and speaks a short confirmation.

After the confirmation finishes, Momo starts a separate music thread. This thread stops any existing music, searches for the requested song using `yt-dlp`, converts the audio using `ffmpeg`, and plays it using `paplay` or `aplay`. The shared state is updated with the song name so the UI can display the music banner.

If the user presses the stop button or says “stop music,” the backend stops the music process and clears the music state.

21 Example Command Flow: Camera and Servo Tracking

The camera and servo system runs continuously in the background. The camera thread captures a frame from the webcam, processes it with OpenCV, and searches for a face. If a face is found, the system calculates where the face is compared to the center of the frame.

If the face is left of center, Momo decreases or increases the servo angle depending on the servo direction setting. If the face is right of center, Momo moves the servo in the opposite direction. The new angle is sent over serial to the Arduino.

The Arduino receives the angle and moves the servo to that position. This causes Momo’s camera to turn toward the person, making the robot feel more interactive and aware.

22 Hardware and Software Integration

Momo is a full-stack hardware and software project. On the software side, it uses Python, Flask, JSON, threading, OpenCV, speech recognition, text-to-speech, requests, subprocesses, and a web interface. On the hardware side, it uses a display, webcam, microphone, speaker, Arduino, wiring, and servo motor.

The backend connects these components together. Flask manages the interface and API routes. The shared state dictionary coordinates the features. Threads allow multiple systems to run at the same time. OpenCV processes camera input. Serial communication connects Python to the Arduino. The Arduino controls the servo motor.

This integration is what allows Momo to respond visually, verbally, and physically.

23 Challenges

One major challenge was designing Momo's physical layout. The wiring, webcam, Arduino, screen, and servo all had to be placed in a way that was functional and organized.

Another challenge was connecting and configuring all the electrical components in the code. Each component needed to work with the backend system correctly.

A major technical challenge was integrating the webcam with the Arduino servo. The camera had to detect a person, calculate the person's position, and send movement commands to the servo.

The most complex software challenge was implementing concurrency and locking. Momo runs many features at the same time, so the code needed a safe way to share data between threads without causing race conditions.

24 What We Learned

This project taught how to build a full-stack hardware and software system. It combined frontend design, backend logic, hardware control, voice interaction, computer vision, and real-time state management.

The project also involved prototyping and computer-aided design because the physical robot had to be designed around real hardware constraints. Placement of the webcam, Arduino, wiring, screen, and servo affected how well the final robot worked.

On the software side, the project taught important engineering concepts such as concurrency, locks, shared state, API routes, JSON persistence, command parsing, and deployment. It also strengthened Python, Flask, and hardware integration skills.

25 More About Concurrency and Locking Were Implemented

Momo uses concurrency so several features can run at the same time without blocking each other. This was implemented with Python background threads. When the program starts, separate daemon threads are created for the timer system, reminder system, camera tracking system, and voice system. The Flask server also runs with threaded request handling, which allows the touch interface and API routes to keep responding while other background features are active.

```
threading.Thread(target=timer_tick, daemon=True).start()
threading.Thread(target=reminder_checker, daemon=True).start()
threading.Thread(target=camera_thread, daemon=True).start()
threading.Thread(target=voice_loop, daemon=True).start()

app.run(host="0.0.0.0", port=5000, debug=False, threaded=True)
```

Each thread has a specific job. The voice thread listens for the wake word, records commands, parses user intent, and triggers actions. The timer thread checks the active timer every second and decreases the countdown. The reminder thread checks every few seconds whether any reminders are due. The camera thread continuously reads webcam frames, detects a person using OpenCV, and sends servo movement commands when needed. The Flask/API thread handles touch interface requests and sends updated state data to the frontend.

All of these threads need to access the same shared data, so Momo uses one central `state` dictionary. This dictionary stores the current screen, voice state, face expression, weather, tasks, timer, reminders, presence status, sleep mode, music status, and servo information. Since multiple threads can read or change this data at the same time, the program uses a lock called `state_lock`.

```
state_lock = threading.Lock()

state = {
    "screen": "clock",
    "voice_state": "idle",
    "face": "idle",
    "weather": {"city": CITY, "temp": "--", "description": "Not
        loaded"},
    "tasks": [],
    "timer": {
        "active": False,
        "seconds_left": 0,
```

```

        "total_seconds": 0,
        "label": "Timer",
    },
    "reminders": [],
    "presence": True,
    "sleep_mode": False,
    "music_song": "",
    "servo": {
        "enabled": SERVO_ENABLED,
        "connected": False,
        "angle": SERVO_START_ANGLE,
    },
}

```

The lock is used with `with state_lock:`. This creates a protected section of code where only one thread can read or update the shared state at a time. For example, when the Flask API sends the current state to the frontend, it first locks the state, copies the values into a snapshot, and then releases the lock. This prevents the frontend from reading data while another thread is halfway through changing it.

```

@app.route("/api/state")
def api_state():
    with state_lock:
        snapshot = {
            "screen": state["screen"],
            "voice_state": state["voice_state"],
            "face": state["face"],
            "weather": dict(state["weather"]),
            "tasks": list(state["tasks"]),
            "timer": dict(state["timer"]),
            "presence": state["presence"],
            "sleep_mode": state["sleep_mode"],
            "servo": dict(state["servo"]),
            "music_song": state["music_song"],
        }
    return jsonify(snapshot)

```

Locking is also used when a feature changes the state. For example, when a timer starts, the backend locks the state, updates the timer data, changes Momo's face to timer mode, and then releases the lock. This prevents another thread from reading an incomplete timer update.

```

with state_lock:

```

```

state["timer"] = {
    "active": True,
    "seconds_left": total,
    "total_seconds": total,
    "label": label,
}
state["face"] = "timer"

```

One important design choice was to avoid holding the lock during slow or blocking operations. Network requests, text-to-speech, music playback, subprocess calls, and camera processing can take time. If the program held `state_lock` during those operations, other threads might freeze while waiting for the lock. To avoid this, Momo does slow work outside the lock, then briefly locks the state only when it needs to store the result.

For example, weather data is fetched before the state is locked. After the request finishes, the program locks the state and updates the weather information.

```

new_weather = fetch_weather()

with state_lock:
    state["weather"] = new_weather
    state["screen"] = "weather"

```

This same idea is used throughout the program: keep locked sections short, only protect shared data, and perform slow operations outside the lock. This makes the robot more responsive because the voice system, camera tracking, timer, reminders, and touch interface can continue working without waiting unnecessarily.

The overall concurrency design allows Momo to behave like a real-time assistant. It can listen for commands, update the display, track a person with the camera, move the servo, run timers, check reminders, and play music at the same time. The locking system keeps these features from interfering with each other by protecting the shared state dictionary from race conditions.

26 Rubric Questions and Project Reflection For Ted Tracy

26.1 How does this project display the aspects of a digital system?

Momo is a strong example of a digital system because it combines input, processing, memory, control logic, communication, and output. The system receives input from voice commands, the touch interface, and the webcam. These inputs are processed by the Flask backend, command parser, OpenCV camera thread, and Arduino servo logic. The project also uses

memory through the shared state dictionary and the JSON task file, which stores task data even after the program restarts. Finally, Momo produces output through the screen interface, spoken responses, music playback, and servo movement.

This matches the structure of a digital system because Momo takes real-world signals, converts them into digital information, processes that information in software, and then controls both software outputs and physical hardware. The webcam detects a person, Python calculates how far the person is from the center of the frame, and the Arduino receives a digital angle command to move the servo. This shows how software and hardware can work together as one interactive system.

26.2 Did we demonstrate technical feasibility?

Yes. The project demonstrates technical feasibility because the major features were implemented and connected into one working system. Momo can display the current time and date, fetch live weather, manage tasks, run Pomodoro timers, set reminders, respond to voice commands, tell jokes, play music, enter sleep mode, detect a person with OpenCV, and move an Arduino-controlled servo toward the user.

The project also shows feasibility because the features are not isolated demos. They communicate through the same Flask backend and shared state dictionary. For example, a voice command can update the shared state, change the screen, trigger a spoken response, and interact with hardware. The touch interface can also control the same backend features. This proves that the system works as an integrated desk assistant rather than separate unrelated components.

26.3 What technologies were involved?

Momo uses both software and hardware technologies. On the software side, the project uses Python, Flask, JSON, threading, OpenCV, speech recognition, text-to-speech, subprocesses, API requests, and a web-based interface. Flask is used as the backend server and provides API routes for the touch interface. JSON is used to save tasks so they persist after restarting. OpenCV is used for webcam-based face and presence detection. Python threading is used so the voice loop, timer, reminders, camera tracking, and Flask server can run at the same time.

On the hardware side, the project uses a display, webcam, microphone, speaker, Arduino, servo motor, and wiring. The Arduino controls the servo motor by receiving angle values from Python over a serial connection. The servo allows Momo to physically turn toward a detected person, making the system more interactive.

26.4 What were the main challenges we had to overcome?

One major challenge was designing Momo's physical layout. The wiring, webcam, Arduino, screen, and servo all had to be placed in a way that was organized, functional, and reliable. Since the webcam needed a clear view and the servo needed enough space to rotate, the hardware placement affected the software performance too.

Another challenge was connecting the electrical components and configuring them in code. The Arduino, servo, webcam, microphone, speaker, and display all had to work with the Python backend. This required testing each component individually and then integrating them into the full system.

A third major challenge was implementing concurrency and locking. Momo has many features running at the same time, including voice listening, timers, reminders, camera tracking, music playback, and the Flask API. Since these features all share the same state dictionary, the project needed `state_lock` to prevent race conditions. The code also had to keep slow operations, such as weather requests, speech playback, and music playback, outside the lock so the program stayed responsive.

26.5 How did we document and explain the concept?

The project is documented through a written system description, architecture diagrams, feature explanations, and poster sections. The documentation explains the main architecture, shared state design, concurrency and locking, touch interface, voice control, weather system, task management, timers, reminders, music playback, camera tracking, and Arduino servo movement.

The poster explains the project visually using a main architecture diagram and a concurrency and locking diagram. The architecture diagram shows how voice, touch, and webcam input move through the Flask backend, shared state, services, and outputs. The concurrency diagram shows how multiple threads safely communicate through the shared state dictionary using `state_lock`. This helps make the system understandable even to someone who has not read the code.

26.6 What does this project lead to next?

Momo could be expanded in several ways. One improvement would be stronger computer vision, such as using a more advanced face tracking model instead of Haar cascades. This could make tracking more accurate in different lighting conditions and from different angles. Another improvement would be better natural language understanding so Momo could handle more complex commands and longer conversations.

The hardware could also be improved. Future versions could include a cleaner enclosure, improved wiring, a stronger servo or pan-tilt mechanism, and more sensors. Momo could

also connect to calendar apps, email, or smart home devices to become a more complete desk assistant. Another possible next step is making the system easier to deploy by packaging the software setup and creating a startup script so Momo runs automatically when powered on.

26.7 What did we learn from the project?

This project taught us how to build a full-stack hardware and software system. We learned how to connect a frontend interface, Flask backend, Python logic, JSON storage, OpenCV vision, speech systems, and Arduino hardware into one working project. We also learned how important prototyping and physical design are when software depends on real hardware placement.

On the software side, we learned about concurrency, locks, shared state, API routes, command parsing, and deployment. We learned that when multiple threads run at the same time, they need a safe way to share data. Using `state_lock` helped prevent race conditions and made the program more reliable. We also learned that slow operations should happen outside locked sections so the system remains responsive.

On the collaboration side, we learned how to divide a complex project into smaller parts, test each part individually, and then integrate everything together. The project required debugging both software and hardware, which showed us the importance of patience, communication, and testing. Overall, Momo helped us better understand how digital systems concepts can be used to build an interactive real-world device.

27 Digital Systems Concepts: Binary, ADC, DAC, and Communication

A digital system is based on information represented as binary values, or 0s and 1s. In Momo, almost every part of the system is handled digitally after the original real-world input is converted into data. Voice, camera input, touch input, weather data, task data, music commands, and servo angles are all represented inside the computer as binary information. The Python backend processes this digital data, stores it in variables and JSON files, sends it through Flask API routes, and communicates with the Arduino using serial messages.

27.1 Binary Data in Momo

Momo uses binary information throughout the system even when the user does not directly see it. For example, a voice command starts as sound waves, but the microphone and speech recognition system convert it into digital audio data and then text. The webcam captures light from the environment, but the camera hardware converts that visual information into digital image frames that OpenCV can process. The touch interface sends button presses

as digital HTTP requests to the Flask backend. The task list is stored digitally in a JSON file, and the shared state dictionary stores values such as the current screen, timer, weather, sleep mode, music status, and servo position.

27.2 ADC and DAC

ADC stands for analog-to-digital conversion. It is used when a real-world analog signal needs to become digital data. Momo relies on ADC indirectly through hardware devices. The microphone uses ADC to convert sound waves into digital audio. The webcam also performs conversion internally by turning light into digital image data. Once converted, Python can process the audio and video as digital information.

DAC stands for digital-to-analog conversion. It is used when digital data needs to become a real-world analog output. Momo relies on DAC indirectly when it speaks or plays music. The text-to-speech system and music playback produce digital audio, and the audio hardware converts that digital signal into analog sound through the speaker.

We did not manually program ADC or DAC in the code. Instead, the microphone, webcam, display/audio system, and operating system drivers handle those conversions. However, ADC and DAC are still important because they allow Momo to connect the physical world with the digital software system.

27.3 Arduino and Servo Communication

Momo communicates with the Arduino using serial communication over USB. The Python backend sends an angle value, such as 90, followed by a newline. The Arduino reads this digital number from the serial port, checks that it is between 0 and 180 degrees, and then moves the servo to that position. The Arduino code uses the `Servo` library, attaches the servo to pin 9, and calls `myServo.write(currentAngle)` to control the motor.

The servo itself is controlled using a PWM-style control signal generated by the Arduino's servo library. This signal tells the servo what angle to move to. In our project, Python decides the desired angle based on either voice commands or OpenCV face tracking, while the Arduino handles the low-level motor control.

27.4 Interrupts, I2C, and SPI

Momo does not directly implement custom interrupts, I2C, or SPI in the project code. The code does not include interrupt service routines, I2C sensor communication, or SPI device communication. Instead, the main hardware communication method is USB serial communication between the Python backend and the Arduino.

The Arduino `Servo` library may use hardware timers internally to generate the servo control signal, but our project code does not manually create or manage interrupts. Similarly,

devices like the webcam, microphone, and audio system may use lower-level communication protocols internally, but those are handled by the hardware, operating system, and device drivers rather than by our code.

27.5 Why This Still Counts as a Digital System

Momo is still a digital system because it takes real-world inputs, converts them into digital data, processes that data using software logic, and produces real-world outputs. The system uses binary data, digital communication, stored state, control logic, and hardware output. The webcam and microphone bring information into the system, Python and Flask process the information, and the screen, speaker, music system, and Arduino-controlled servo produce the outputs. This shows the full path of a digital system: input, digital processing, memory, control, and output.

28 Conclusion

Momo Desk Buddy is an interactive desktop assistant that combines productivity features with robot behavior. It can show the time, fetch weather, manage tasks, run Pomodoro timers, set reminders, give daily summaries, tell jokes, play music, enter sleep mode, detect a person with OpenCV, and turn toward the user using an Arduino-controlled servo.

The strongest part of the project is its architecture. A Flask backend controls the system, a shared state dictionary connects the features, and background threads allow voice, timers, reminders, camera tracking, music, and the UI to run at the same time. Locking keeps the shared data safe, while the hardware systems make Momo feel like a real desk companion.